

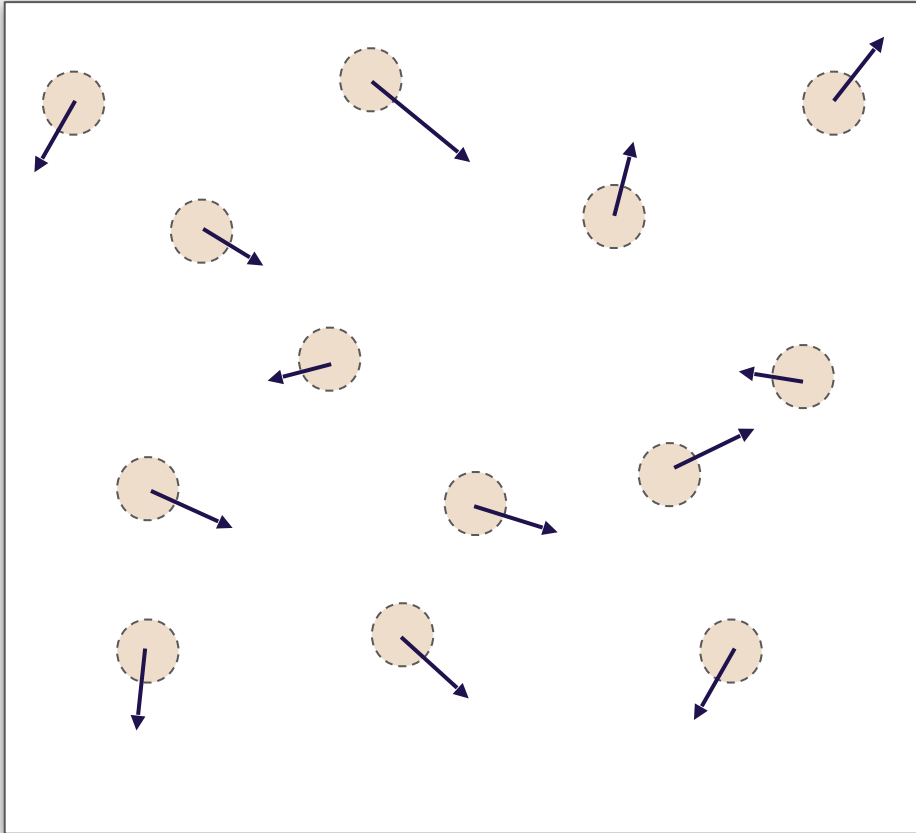
Объектно-ориентированное программирование

Никита Остроухов

Кафедра информатики МФТИ

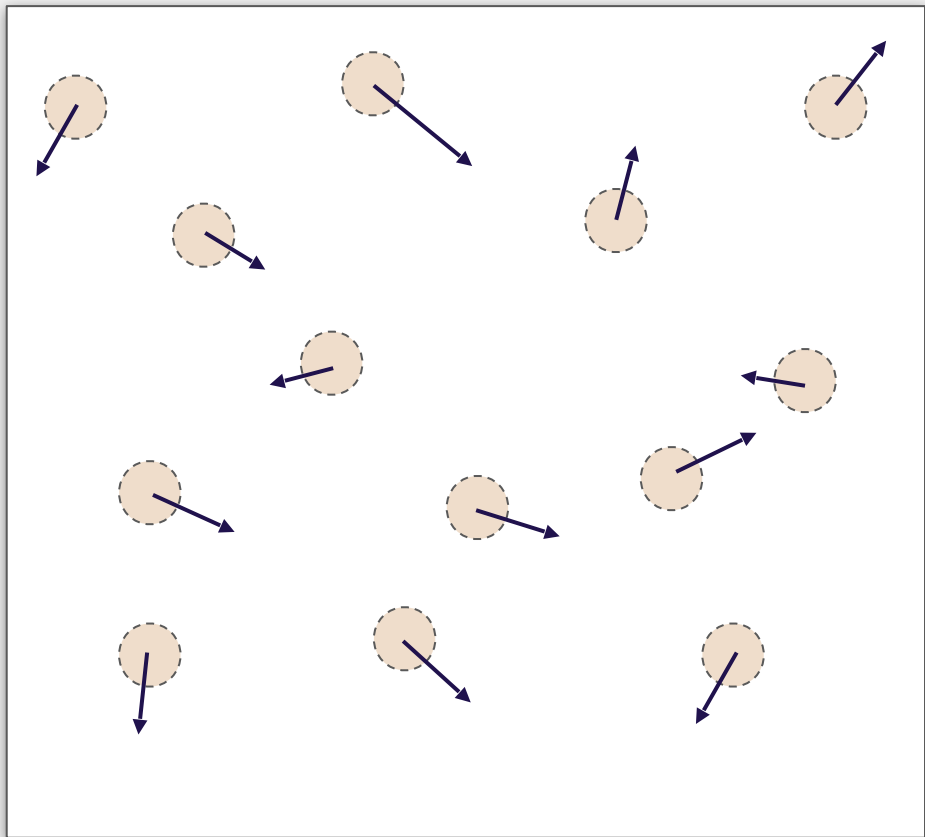
Идеальный газ

Идеальный газ



Модель состоит из **сотен шаров**, каждый из которых **обладает** своим набором **координат** и **скоростей**

Идеальный газ



**У нас уже есть функции рисования, движения
и столкновения шаров**

Идеальный газ

```
for (;;)
{
```

...

```
}
```

Если мы попробуем **расширить** нашу
прошлую **модель** из нескольких шаров
до сотни

Идеальный газ

```
// Объявляем 100 структур
```

```
for (;;)
{
```

```
...
```

```
}
```

Нам придется написать **сто переменных**
структуры

Идеальный газ

```
// Объявляем 100 структур
```

```
for (;;)
{
    drawBall(ball1);
    ...
    drawBall(ball100);

    ...
}
```

Сто раз продублировать код вызова
функции drawBall

Идеальный газ

```
// Объявляем 100 структур
```

```
for (;;)
{
```

```
...
```

```
    moveBall(&ball1, dt);
```

```
    ...
```

```
    moveBall(&ball100, dt);
```

```
...
```

```
}
```

Сто раз **продублировать** код вызова
функции **moveBall**

Идеальный газ

// Объявляем 100 структур

```
for (;;)
{
    if (isCollision(ball1, ball2))
        resolveCollision(&ball1, &ball2);
    ...
    if (isCollision(ball1, ball100))
        resolveCollision(&ball1, &ball2);

    if (isCollision(ball2, ball3))
        resolveCollision(&ball1, &ball2);
    ...
    if (isCollision(ball2, ball100))
        resolveCollision(&ball1, &ball2);
    ...
}
```

И $99 + 98 + \dots + 1$ раз == **2500 раз (!!!)**
написать проверку столкновений

Как избежать этого?

Массивы структур

Массивы

```
Ball ball1;  
Ball ball2;  
...  
Ball ball100;
```



С каких это пор **это**...

Массивы

```
Ball balls[100];
```



С каких это пор **это**...
стало **лучше чем** это?

Массивы

```
Ball balls[100];
```



Это массив структур из ста элементов
типа **Ball**

Массивы

```
Ball balls[100];
```

```
for (;;)
{
```

```
    drawBall(balls[0]);
```

```
    ...
```

```
    drawBall(balls[99]);
```

```
    ...
```

```
}
```

Обращение к элементу массива

Массивы

```
Ball balls[100];  
  
for (;;)   
{  
    for (int i = 0; i < 100; i++)  
        drawBall(balls[i]);  
  
    ...  
  
}
```

Тот же код **с использованием цикла**

Массивы

```
for (;;)
{
    for (int i = 0; i < 100; i++)
    {
        drawBall(balls[i]);
        moveBall(balls[i]);

        for (int j = i; j < 100; j++)
        {
            if (i == j)
                continue;

            if (isCollision(balls[i], balls[j]))
                resolveCollision(&balls[i],
                                &balls[j]);
        }
    }
}
```

**Идеальный газ с использованием
массива структур**

Массив - удобный способ
хранить данные

Оператор continue

```
for (;;)
{
    for (int i = 0; i < 100; i++)
    {
        drawBall(balls[i]);
        moveBall(balls[i]);

        for (int j = i; j < 100; j++)
        {
            if (i == j)
                continue;

            if (isCollision(balls[i], balls[j]))
                resolveCollision(&balls[i],
                               &balls[j]);
        }
    }
}
```



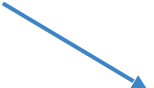
Оператор пропускающий текущую **итерацию**
в цикле

Оператор continue

```
for (;;)
{
    for (int i = 0; i < 100; i++)
    {
        drawBall(balls[i]);
        moveBall(balls[i]);

        for (int j = i; j < 100; j++)
        {
            if (i == j)
                continue;

            if (isCollision(balls[i], balls[j]))
                resolveCollision(&balls[i],
                               &balls[j]);
        }
    }
}
```



Распространяет свое действие только на тот цикл в котором был написан

Операторы цикла

```
for (;;)
{
    for (int i = 0; i < 100; i++)
    {
        drawBall(balls[i]);
        moveBall(balls[i]);

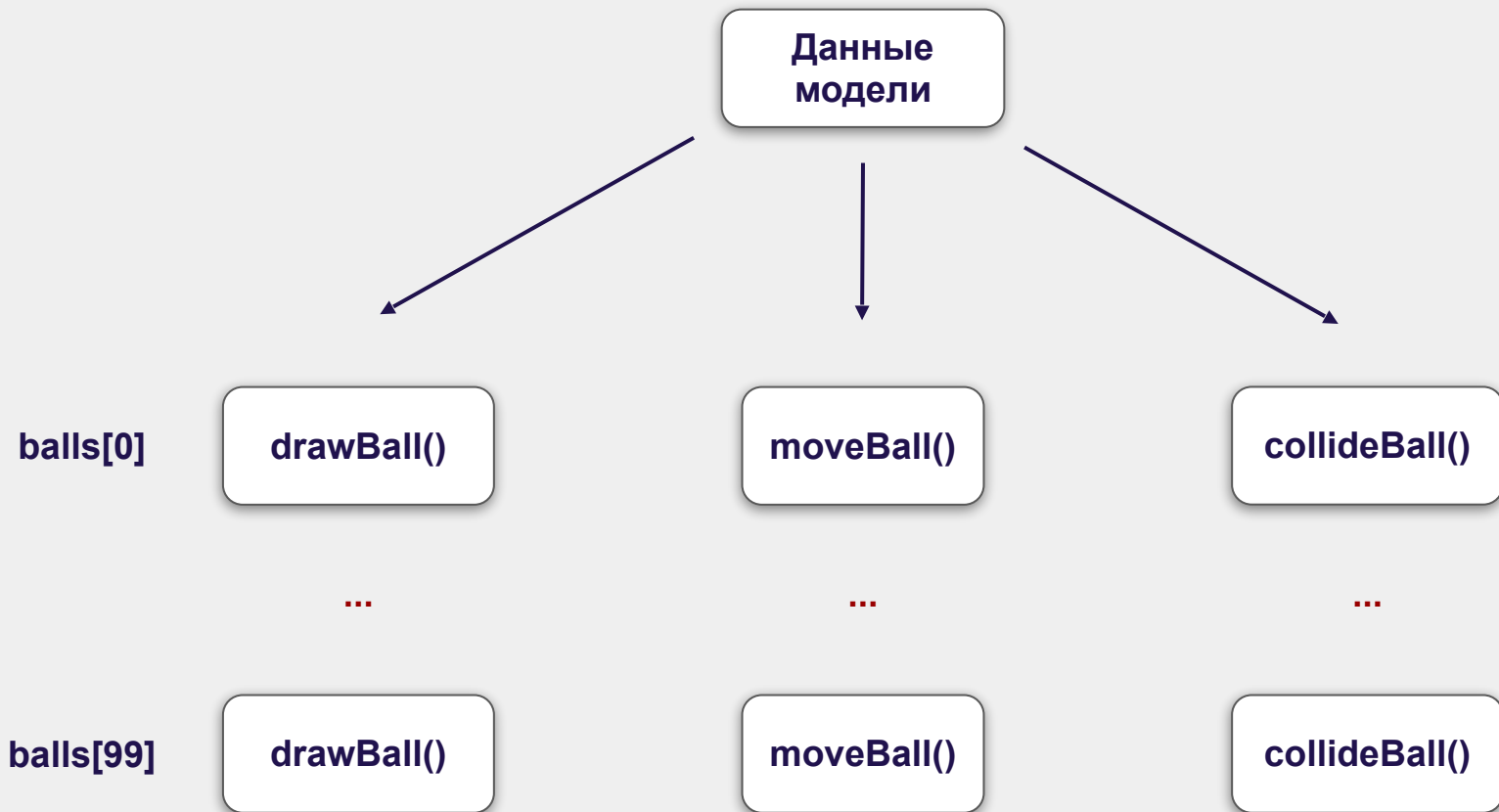
        for (int j = i; j < 100; j++)
        {
            if (i == j)
                continue;

            if (isCollision(balls[i], balls[j]))
                resolveCollision(&balls[i],
                                &balls[j]);
        }
    }
}
```

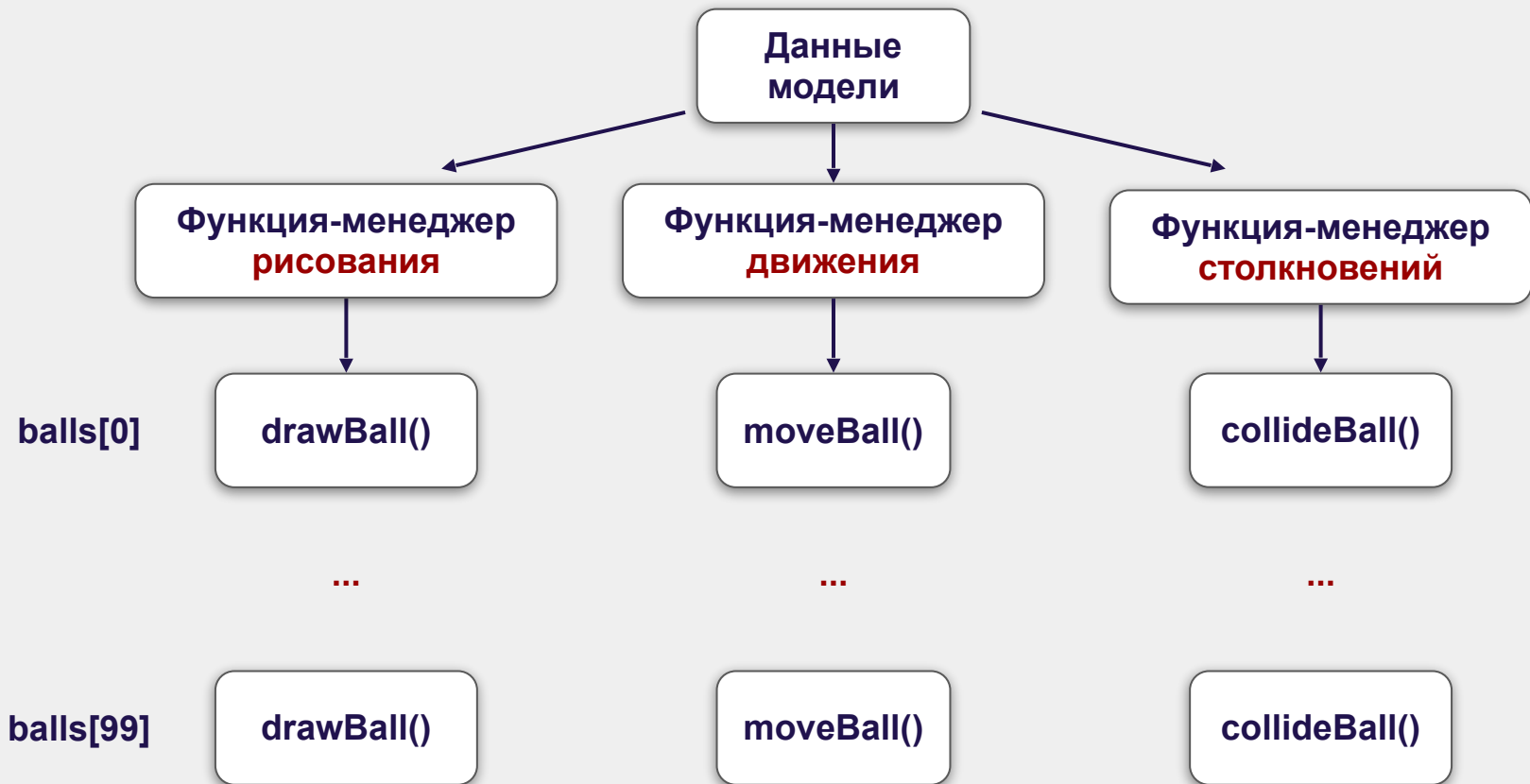
Операторы в цикле:

break
continue

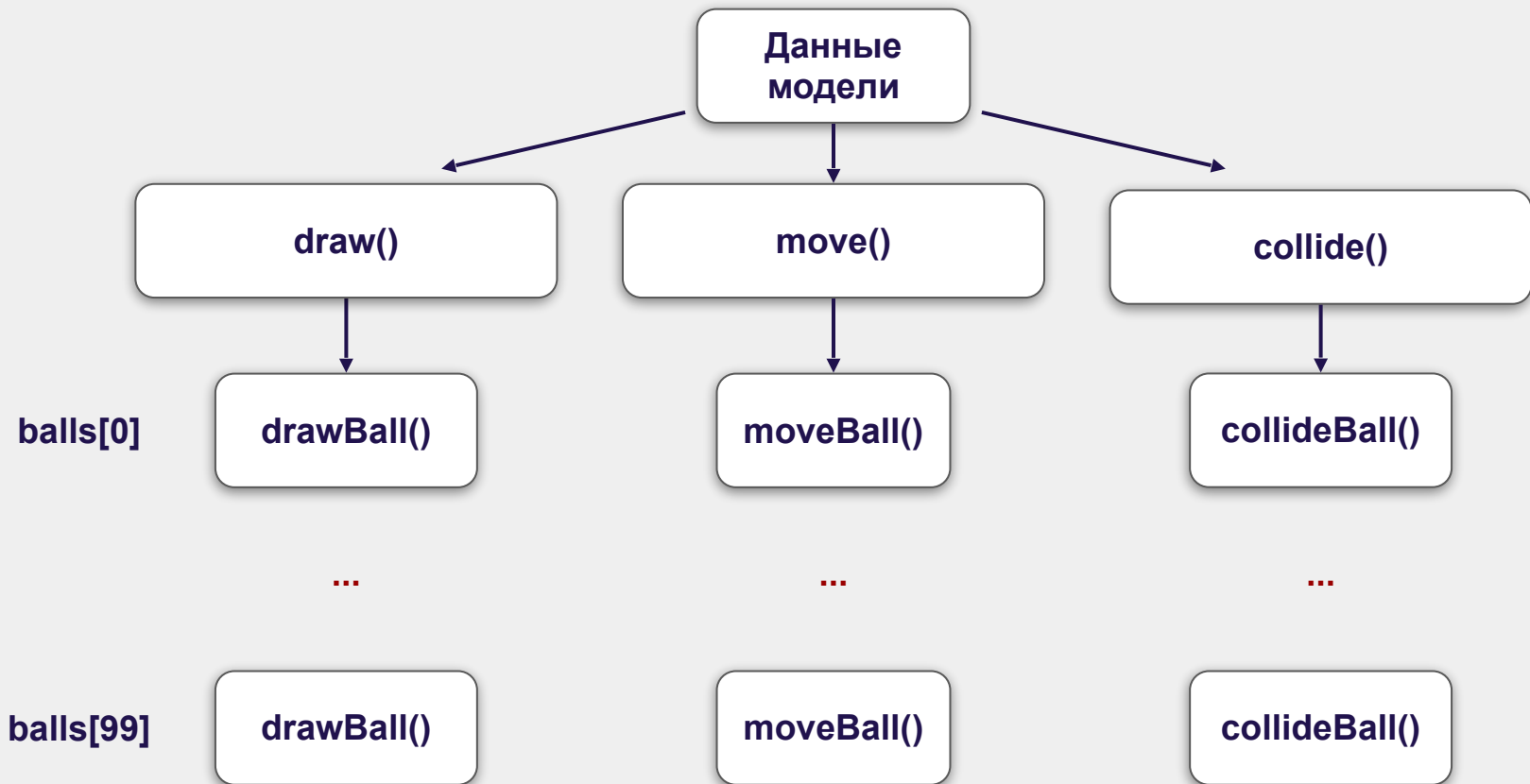
Архитектура



Архитектура



Архитектура



Массивы

```
void draw (Ball* balls, int count);  
void move (Ball* balls, int count);  
void collide (Ball* balls, int count);
```

Функции-менеджеры принимают **массив**
в качестве **аргумента**

Массивы

```
void draw (Ball* balls, int count);  
void move (Ball* balls, int count);  
void collide (Ball* balls, int count);
```



Почему тип массива **совпадает** с типом
адреса структуры **Ball**?

Массив в памяти

Оперативная память



↑
0xFF32E6A6

```
struct Ball  
{  
    int x;  
    int y;  
    float vx;  
    float vy;  
};
```

...

```
Ball ball1 = {5, 3, 2.0f, 1.0f};
```

Объявление структуры

Оперативная память



↑
0xFF32E6A6

```
struct Ball
{
    int x;
    int y;
    float vx;
    float vy;
};
```

...

```
Ball ball1 = {5, 3, 2.0f, 1.0f};
```

←
**Адрес этой структуры в памяти
&ball1**

Оперативная память



0xFF32E6AA



Адрес поля “y” структуры
&ball.y

```
struct Ball
```

```
{
```

```
    int x;
```

```
    int y;
```

```
    float vx;
```

```
    float vy;
```

```
};
```

```
...
```

```
Ball ball1 = {5, 3, 2.0f, 1.0f};
```

Оперативная память



0xFF32E6A6

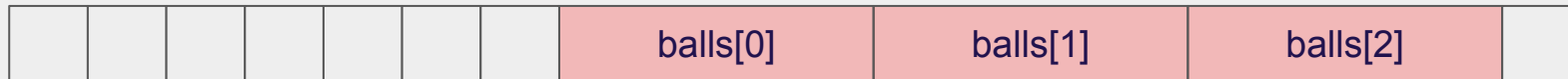
```
struct Ball
{
    int x;
    int y;
    float vx;
    float vy;
};
```

...

```
Ball ball1 = {5, 3, 2.0f, 1.0f};
```

Структура представлена в памяти как
поряд лежащие поля этой структуры

Оперативная память



0xFF32E6A6

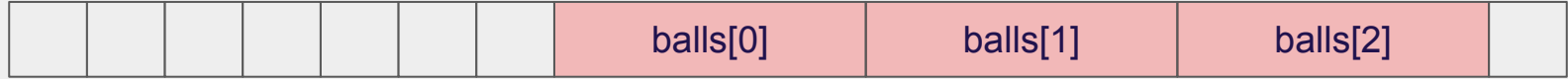
```
struct Ball
{
    int x;
    int y;
    float vx;
    float vy;
};
```

...

```
Ball balls[3];
```

Массив представлен как подряд лежащие **элементы** массива

Оперативная память



0xFF32E6A6

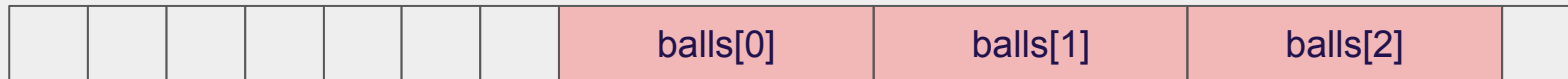
```
struct Ball
{
    int x;
    int y;
    float vx;
    float vy;
};
```

...

```
Ball balls[3];
```

Массив - это **адрес** его первого элемента

Оперативная память

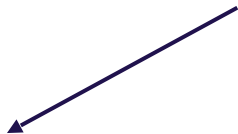


0xFF32E6A6

```
struct Ball
{
    int x;
    int y;
    float vx;
    float vy;
};
```

...

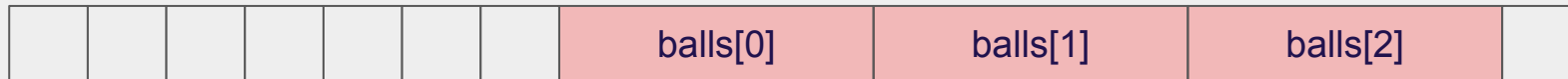
```
Ball balls[3];
```



Массив - это **адрес** его первого элемента

Переменная **balls** совпадает с адресом **первого элемента** в этом массиве

Оперативная память



0xFF32E6A6

```
struct Ball
{
    int x;
    int y;
    float vx;
    float vy;
};
```

...

Ball balls[3];

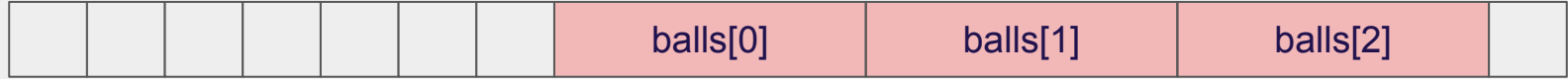


Массив - это **адрес** его первого элемента

Переменная **balls** совпадает с адресом **первого элемента** в этом массиве

Элемент массива имеет тип **Ball**

Оперативная память



0xFF32E6A6

```
struct Ball
{
    int x;
    int y;
    float vx;
    float vy;
};
```

...

```
Ball balls[3];
```

Переменная **balls** имеет тип **Ball***

Функции-менеджеры

```
void draw(Ball* balls, int count)
{

}
```

Напишем реализацию одной из функций

Функции-менеджеры

```
void draw(Ball* balls, int count)
{
    for (int i = 0; i < count; i++)
        drawBall(balls[i]);
}
```

Напишем реализацию одной из функций

Функции-менеджеры

```
void draw(Ball* balls, int count)
{
    for (int i = 0; i < count; i++)
        if (balls[i].isVisible)
            drawBall(balls[i]);
}
```

Мы можем добавить структуре Ball
новое поле **isVisible**

Задание

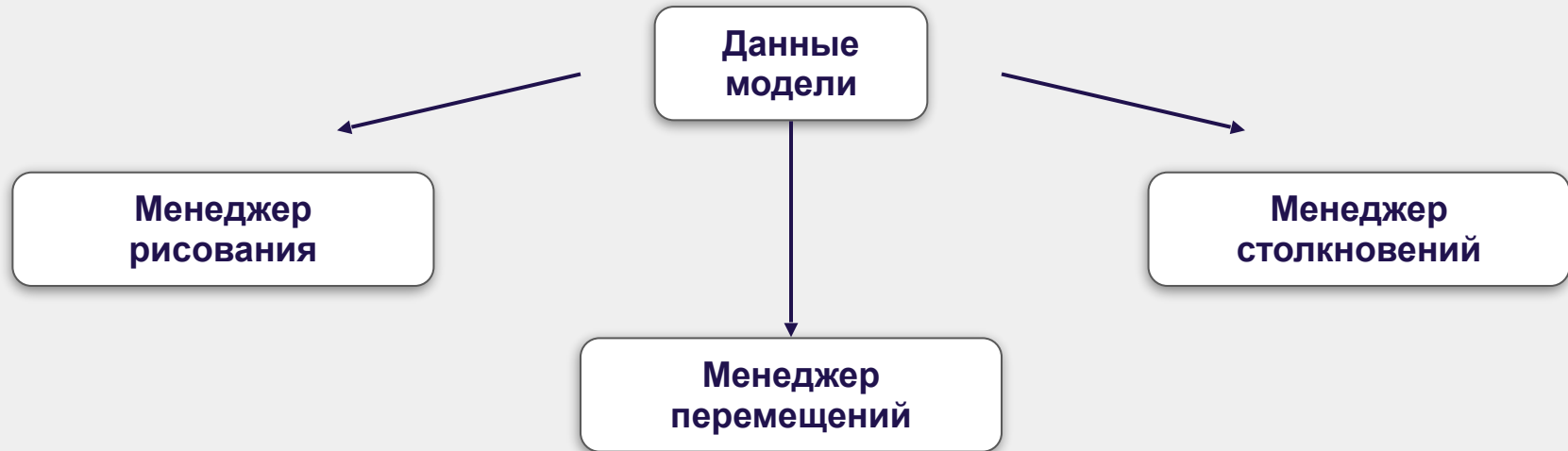
Реализовать остальные функции-менеджеры

Функции-менеджеры

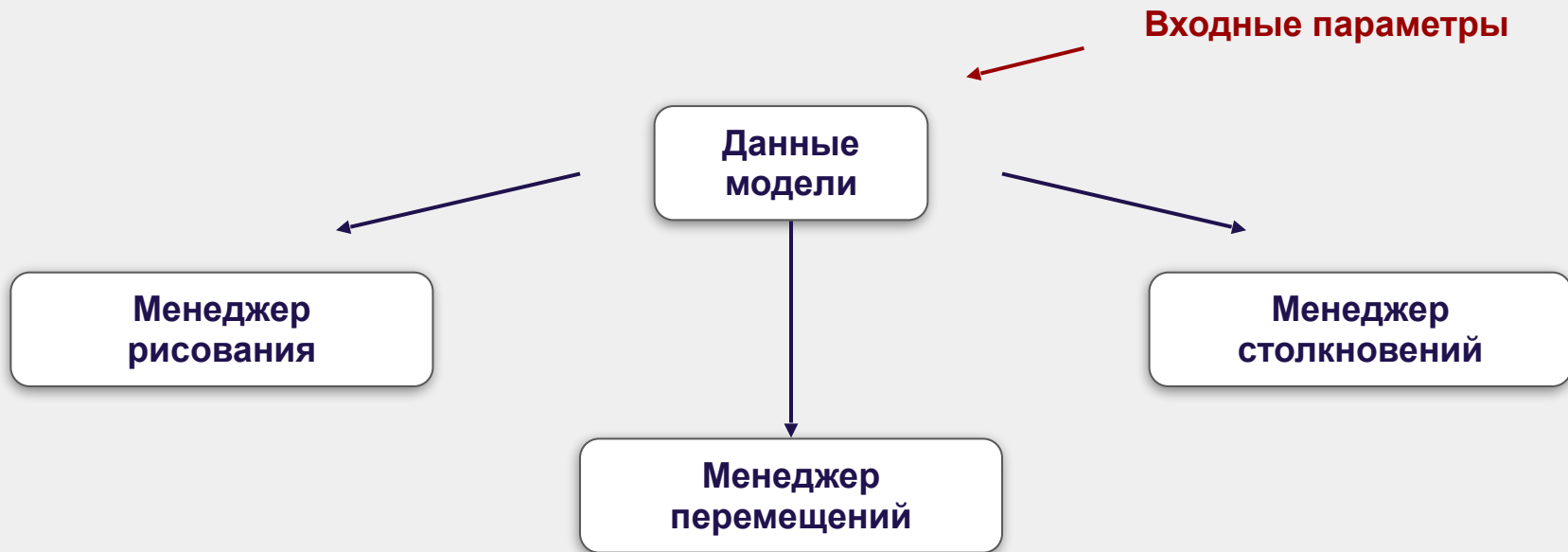
```
...  
  
int main()  
{  
    Ball balls[100];  
  
    // Задание начальных условий  
  
    for (;;)   
    {  
        draw(balls, 100);  
        move(balls, 100);  
        collide(balls, 100);  
    }  
  
    return 0;  
}
```

Функции-менеджеры существенно упрощают работу с **массивами** данных

Архитектура



Архитектура



Мы реализовали нано-движок